

Deep Algorithm Playbook – World Class URL Shortener

1) Code generation and collisions

Snowflake style ID (timestamp | region | worker | sequence) to Base62 or Base58. Hash based: code = base62(HMAC_SHA256(url, pepper))[0..k]; on hit, rehash with salt and counter. Custom alias: check reserved or abusive regex, then compare and set. Anti guessing: length floor plus mixed alphabet; rotate pepper on breach.

2) Redirect (read) algorithm

Tiered lookup: edge, then regional Redis, then durable store. Singleflight per key; negative caching (404 or blocked) with short TTL. SWR (stale while revalidate) at edge; return cached 302 plus async refresh.

3) Write, update and read after write

Idempotency key equals hash of body to deduplicate retries. Two step: write durable (version increment), then publish change event; caches do last write wins on version. Read after write priming: writer pushes to Redis and nearest edges before acknowledgement.

4) Cache population and propagation

Snapshot plus tail: edges boot with top K snapshot per tenant, then tail compacted change log. Warm promotion: if key crosses requests per second threshold, push to edges nearest demand.

5) Hot key detection

Count Min Sketch plus Space Saving to track heavy hitters per region; decay with EMA; promote or replicate.

6) Eviction and TTL

W TinyLFU or LFU for Redis; probabilistic TTL jitter to avoid herd expiry. Early refresh when age is greater than threshold times TTL with token bucket cap. Timing wheel for mass expirations (scheduled deletes).

7) Rate limiting and abuse control

Token Bucket (burst) plus Sliding Window counters (sustained) per IP, tenant or domain. Bot heuristics: user agent parsing, headless signals, challenge ladder; reputation cache for deny lists. Safe browsing: hash prefix lookup with positive result caching.

8) Analytics aggregation

HLL for uniques; t digest or DDSketch for latency and geo distances; reservoir sampling for long tail refs. Rollups per tenant, domain, code, minute or hour with watermarking for late events.

9) Sharding and routing

Rendezvous or Consistent hashing with virtual nodes; hash tags to co locate tenant shards. Graceful reshard: move minimal key ranges; dual write during cutover.

10) Ordering, deduplication, exactly once

Per key sequence in change events; consumers apply idempotent upserts. Kafka partition by key; out of order window plus tombstones for deletes.

11) Backpressure and resilience

Circuit breaker plus AIMD concurrency to origin; shed with 429 or 503 plus Retry After. Adaptive timeouts (P99 aware) and jittered retries (exponential).

12) URL normalization and safety

Canonicalize: trim, lower host, IDNA or punycode, strip default ports, sort query, block dangerous schemes. Open redirect guard for custom domains; CSP and HSTS headers prebuilt in cached response.

13) API Design

POST /shorten – Create a short URL.

Request: { 'url': 'https://example.com', 'custom_alias': 'myalias', 'expiry': '2025-12-31' }

Response: { 'short_url': 'https://sho.rt/myalias', 'code': 'myalias' }

GET /:code – Redirect to the original URL.

Response: 301 or 302 redirect.

GET /:code/info – Retrieve details about a short URL.

Response: { 'url': 'https://example.com', 'created_at': '...', 'clicks': 123 }

DELETE /:code – Delete or deactivate a short URL.

Response: { 'status': 'deleted' }

GET /analytics/:code – Retrieve analytics data.

Response: { 'total_clicks': 1000, 'unique_visitors': 900, 'top_countries': [...] }

All APIs should include authentication headers for user-specific data, rate limiting headers in responses, and appropriate error codes for invalid requests or unauthorized access.